

POZNAŃ UNIVERSITY OF TECHNOLOGY

FACULTY OF CONTROL, ROBOTICS
AND ELECTRICAL ENGINEERING

INSTITUTE OF ROBOTICS AND MACHINE INTELLIGENCE

DIVISION OF CONTROL AND INDUSTRIAL ELECTRONICS



FINAL MICROPROCESSOR SYSTEMS TASK

MICROPROCESSOR SYSTEMS

LABORATORY REPORT

OSKAR JABŁONOWSKI 159568

JAKUB KAŻMIERCZAK 159576

JAKUB KRUSICKI 159565

INSTRUCTOR:

TEODOR KRUPSKI, M.SC

TEODOR.KRUPSKI@PUT.POZNAN.PL

23.02.2026

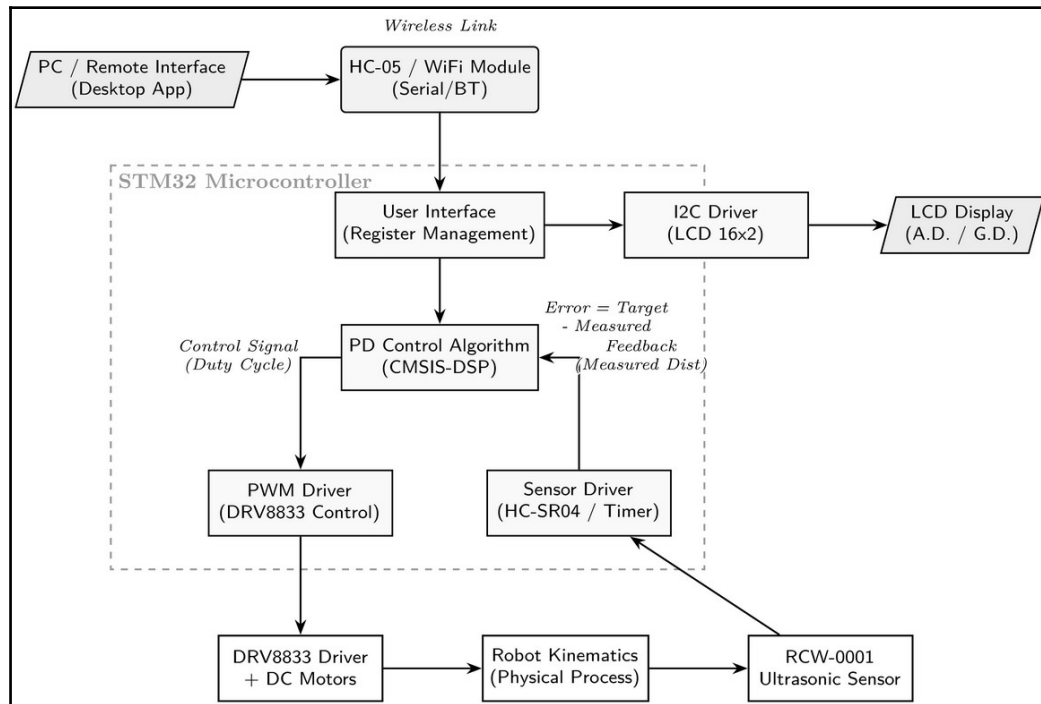


Contents

0	Introduction	3
1	Project	
1.1	Specification	4
1.2	Implementation	5
1.3	Test results	10
1.4	Conclusion	10
2.	Additional Requirements Met	
2.1	Use of a version-control system	11
2.2	System ensures steady-state error at 1% of control range or less.	11
2.3	Dedicated desktop application as a graphical user interface.	12
2.4	Additional user output device - LCD Display.	13
2.5 & 2.6	Additional user input device and Use of Network Communication	13
3	References	14

0. INTRODUCTION

This laboratory project presents the design and implementation of a microprocessor-based control and measurement system in the form of a mobile robotic car capable of regulating its distance from a wall. The system operates as a closed-loop control system in which distance measurements are continuously acquired and used to adjust motor speed.



Hardware platform:

- STM32 microcontroller board (NUCLEO series)
- RCW-0001 ultrasonic (supersonic) distance sensor
- DRV8833 dual H-bridge motor driver (1.5 A, 10 V)
- Two DC motors (1:48 gearbox, 200 RPM, 3–6 V)
- Two battery baskets:
 - 3×AA battery pack
 - 4×AA battery pack
- Buck converter regulating motor voltage to 6 V
- Separate stabilized 5 V supply for STM32 board
- modHC-05 (FC-14) Bluetooth/WiFi communication module
- LCD display for real-time system information

Software environment:

- STM32CubeIDE
- HAL and CMSIS libraries
- Timer based PWM motor control
- Closed loop distance regulation algorithm
- Wireless monitoring and parameter configuration

1.1 SPECIFICATION

The goal of the system is to maintain a predefined distance between the robot and a wall using automatic regulation.

Minimum requirements:

- System performs cyclic, synchronous measurement of process value (controlled variable) with constant sample time.
- System allows for process control in safe range of process value.
- System ensure steady-state error at 5% of the control range
- System allows to set a reference value (set-point) with serial port or other UI,
- System allows reading current values of: measurement, reference and control signals with serial port or other UI,

Additional requirements:

- Use of a version-control system.
- System ensure steady-state error at 1% of the control range
- Dedicated desktop application as a graphical user interface
- Additional user output device - LCD
- Additional user input device
- Use of network communication

1.2 IMPLEMENTATION

Hardware Implementation

The STM32 microcontroller serves as the central control unit. The RCWL0001 ultrasonic sensor measures distance by transmitting an acoustic pulse and measuring echo return time. To interface with this sensor, a dedicated driver module was implemented, as shown in the code below:

```
#ifndef INC_HC_SR_04_H_
#define INC_HC_SR_04_H_
#include "main.h"

typedef uint32_t TIM_Channel;

struct us_sensor_str
{
    TIM_HandleTypeDef *htim_echo;
    TIM_HandleTypeDef *htim_trig;
    TIM_Channel trig_channel;

    volatile uint32_t distance_cm;
};

void hc_sr04_init(struct us_sensor_str *us_sensor, TIM_HandleTypeDef
*htim_echo, TIM_HandleTypeDef *htim_trig, TIM_Channel channel);
uint32_t hc_sr04_convert_us_to_cm(uint32_t distance_us);

#endif /* INC_HC_SR_04_H_ */
```

hc_sr_04.h file.

```
#include "hc_sr_04.h"

#define HC_SR04_US_TO_CM_CONVERTER 58

void hc_sr04_init(struct us_sensor_str *us_sensor, TIM_HandleTypeDef
*htim_echo, TIM_HandleTypeDef *htim_trig, TIM_Channel trig_channel)
{
    us_sensor->htim_echo = htim_echo;
    us_sensor->htim_trig = htim_trig;
    us_sensor->trig_channel = trig_channel;
}
```

```
    HAL_TIM_IC_Start_IT(us_sensor->htim_echo, TIM_CHANNEL_1 |  
TIM_CHANNEL_2);  
    HAL_TIM_PWM_Start(us_sensor->htim_trig, us_sensor->trig_channel);  
}  
  
uint32_t hc_sr04_convert_us_to_cm(uint32_t distance_us)  
{  
    return (distance_us / HC_SR04_US_TO_CM_CONVERTER);  
}
```

hc_sr_04.c file

The driver implementation utilizes the STM32's timer peripherals in two different modes to handle the sensor's timing requirements. The *hc_sr04_init()* function configures one timer for PWM generation to produce the trigger pulse, while a second timer operates in input capture mode with interrupts enabled on both channels to precisely measure the echo pulse duration. This approach ensures accurate distance measurements by leveraging hardware timing capabilities rather than software-based timing, which could introduce latency and measurement errors.

The sensor's measurement principle relies on the speed of sound, which travels approximately 58 microseconds per centimeter round-trip. This relationship is captured in the *HC_SR04_US_TO_CM_CONVERTER* macro, and the *hc_sr04_convert_us_to_cm()* function implements the conversion from the measured echo time in microseconds to distance in centimeters.

The custom *us_sensor_str* data structure encapsulates all necessary hardware handles and stores the computed distance value. The volatile qualifier on the *distance_cm* member ensures proper behavior when this value is accessed from both interrupt contexts and the main program flow, preventing compiler optimizations that could lead to stale data reads.

This modular driver design allows for easy integration with the main application code and provides a clean abstraction layer between the low-level timer hardware and the higher-level distance measurement functionality.

Motor control is implemented using the DRV8833 motor driver. Two DC motors are driven using PWM signals generated by STM32 timers.

Power System:

- Motors powered from dual AA battery baskets regulated to 6 V via buck converter.
- STM32 powered independently using regulated 5 V supply to ensure stable logic operation.

Communication:



The modHC05 (FC14) module enables wireless communication used for:

- monitoring measured distance,
- changing reference distance,
- debugging and diagnostics.

User Interface:

The system provides real-time feedback through an alphanumeric LCD display, presenting the current measured distance from the ultrasonic sensor alongside the target distance set-point. To interface with the LCD module, a driver was implemented using the I2C communication protocol.

The core of the LCD communication is handled by two key functions that manage the 4-bit data transfer protocol over I2C:

```
// Function to send a command to the LCD
HAL_StatusTypeDef lcd_send_cmd(uint8_t cmd)
{
    uint8_t data_u, data_l;
    uint8_t data_t[4];

    data_u = (cmd & 0xF0);           // Upper nibble
    data_l = ((cmd << 4) & 0xF0);    // Lower nibble

    data_t[0] = data_u | 0x0C;       // Enable high, RS Low
    data_t[1] = data_u | 0x08;       // Enable Low, RS Low
    data_t[2] = data_l | 0x0C;       // Enable high, RS Low
    data_t[3] = data_l | 0x08;       // Enable Low, RS Low

    return HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD << 1, data_t, 4,
    HAL_MAX_DELAY);
}

// Function to send data to the LCD
HAL_StatusTypeDef lcd_send_data(uint8_t data)
{
    uint8_t data_u, data_l;
    uint8_t data_t[4];

    data_u = (data & 0xF0);           // Upper nibble
    data_l = ((data << 4) & 0xF0);    // Lower nibble

    data_t[0] = data_u | 0x0D;       // Enable high, RS high
    data_t[1] = data_u | 0x09;       // Enable Low, RS high
    data_t[2] = data_l | 0x0D;       // Enable high, RS high
    data_t[3] = data_l | 0x09;       // Enable Low, RS high

    return HAL_I2C_Master_Transmit(&hi2c1, SLAVE_ADDRESS_LCD << 1, data_t, 4,
    HAL_MAX_DELAY);
}
```

```
}
```

These functions construct 4-byte sequences that properly manage the Enable (EN) and Register Select (RS) control lines through the I2C interface. The data is split into upper and lower nibbles, with the Enable pin toggled to latch each nibble into the LCD controller. The key difference between the two functions lies in the RS bit setting: commands use RS=0 (control operations), while data uses RS=1 (character display).

The initialization sequence follows the HD44780 controller's timing requirements:

```
void lcd_init(void)
{
    // 4-bit initialization sequence
    HAL_Delay(50);  lcd_send_cmd(0x30);  // Wait >40ms
    HAL_Delay(5);   lcd_send_cmd(0x30);  // Wait >4.1ms
    HAL_Delay(1);   lcd_send_cmd(0x30);  // Wait >100us
    HAL_Delay(10);  lcd_send_cmd(0x20);  // Switch to 4-bit mode

    // Display configuration
    lcd_send_cmd(LCD_CMD_FUNCTION_SET);  // 2-line, 5x8 font
    lcd_send_cmd(LCD_CMD_DISPLAY_ON);    // Enable display
    lcd_send_cmd(LCD_CMD_CLEAR_DISPLAY);  // Clear screen
    lcd_send_cmd(LCD_CMD_ENTRY_MODE_SET); // Auto-increment cursor
}
```

For presenting the measurement data, a dedicated display function formats the information:

```
void distance_display(uint32_t *giv_dist, uint32_t *acc_dist)
{
    lcd_clear();

    // First line - Given (measured) distance
    lcd_put_cursor(0,0); lcd_send_string("G.D.");
    lcd_put_cursor(0,6); lcd_send_string((char*)giv_dist);
    lcd_put_cursor(0,10); lcd_send_string("cm");

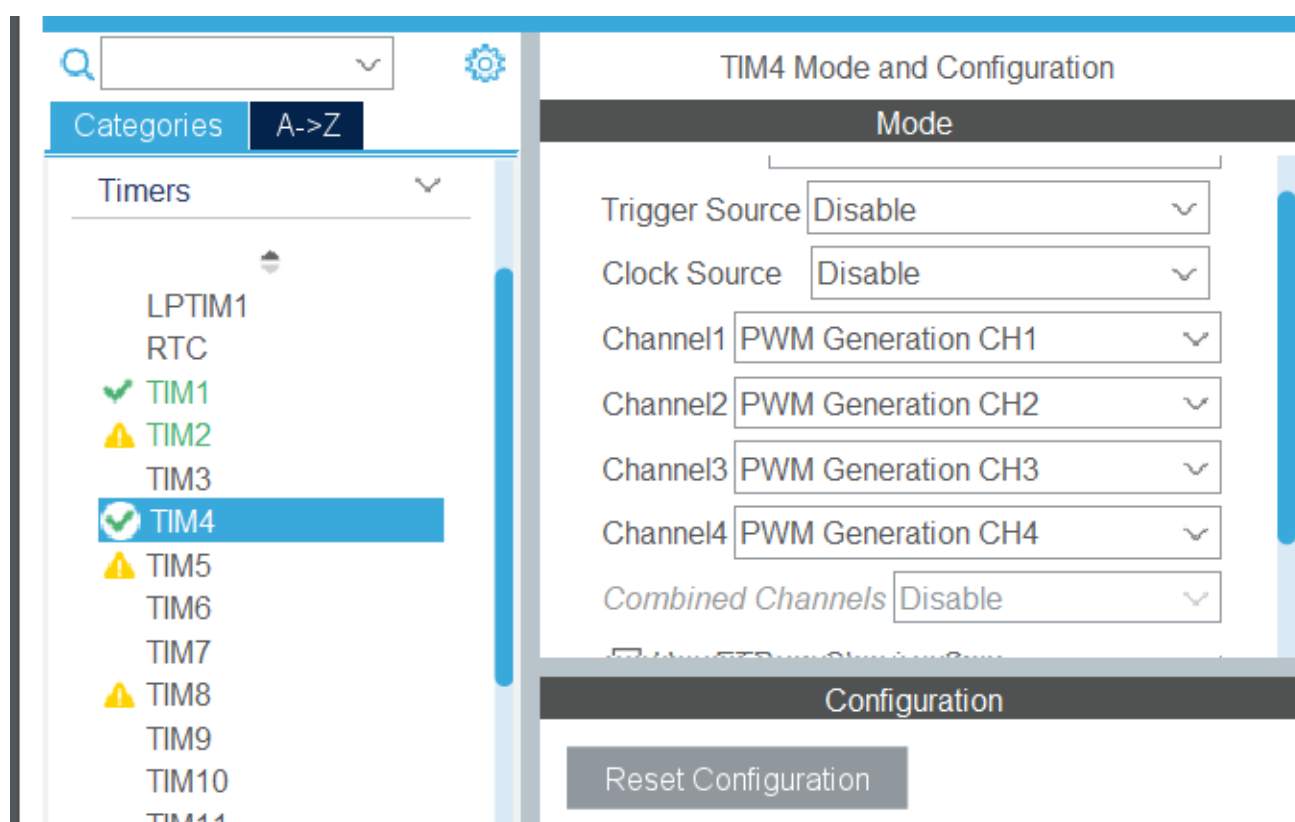
    // Second line - Accepted (target) distance
    lcd_put_cursor(1,0); lcd_send_string("A.D.");
    lcd_put_cursor(1,10); lcd_send_string("cm");

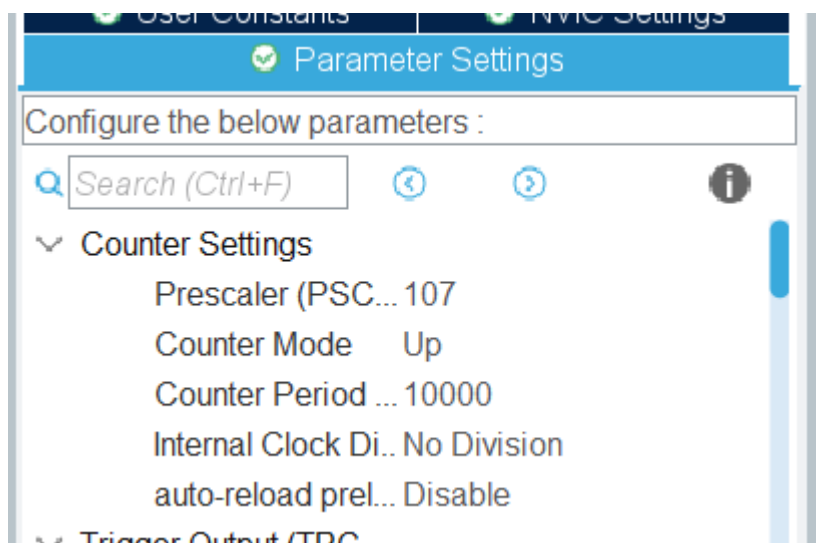
    char buffer[12];
    itoa(*acc_dist, buffer, 10);
    lcd_put_cursor(1,6); lcd_send_string(" "); // Clear previous value
    lcd_put_cursor(1,6); lcd_send_string(buffer);
}
```

```
HAL_Delay(150); // Controlled update rate
}
```

This implementation creates an intuitive two-line display format, with the first line showing the real-time measured distance ("G.D.") and the second line displaying the target set-point ("A.D."). The 150ms update delay provides stable visual feedback without excessive screen flickering.

PWM generation is performed using hardware timers to ensure precise speed control and reduced error margin.



**motor control logic:**

The implemented motor control logic operates in a 50ms loop, utilizing a Proportional-Derivative (PD) controller to compute the required speed based on the robot's distance to a target. If the robot is within a defined distance tolerance, the output is zeroed to prevent jitter. Otherwise, the PD output is passed to a motor driver function that clamps the value to ensure it strictly falls within the physical minimum (**MIN_PWM**) and maximum (**PWM_MAX**) operational limits. Finally, this constrained PWM value is routed to four hardware timer channels,

driving the motors either forward or backward depending on whether the calculated speed is positive or negative.

```
128
129 void SetRobotSpeed(int32_t speed)
130 {
131     if (speed > PWM_MAX) speed = PWM_MAX;
132     if (speed < -PWM_MAX) speed = -PWM_MAX;
133
134     if (speed > 0 && speed < MIN_PWM) speed = MIN_PWM;
135     if (speed < 0 && speed > -MIN_PWM) speed = -MIN_PWM;
136
137     if (speed == 0) {
138         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, 0);
139         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, 0);
140         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
141         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, 0);
142         return;
143     }
144
145     if (speed > 0) {
146         // Przód
147         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, speed); // L Przód
148         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, 0);
149         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, speed); // R Przód
150         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, 0);
151     } else {
152         // Tyż
153         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, 0);
154         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, -speed); // L Tyż
155         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
156         __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, -speed); // R Tyż
157     }
158 }
159
```

```
161 float Calculate_PD(PD_Controller *pd, uint32_t *current_distance)
162 {
163     uint32_t now = HAL_GetTick();
164     float dt = (now - pd->last_time) / 1000.0f;
165     if (dt <= 0.0f) dt = 0.001f;
166
167     float current_distance_f = *current_distance;
168     float error = current_distance_f - pd->target_distance;
169
170     if (error > -TOLERANCE_CM && error < TOLERANCE_CM){
171         pd->prev_error = error;
172         pd->last_time = now;
173
174         return 0.0f;
175     }
176
177     float derivative = (error - pd->prev_error) / dt;
178
179     float output = (pd->Kp * error) + (pd->Kd * derivative);
180
181     pd->prev_error = error;
182     pd->last_time = now;
183
184     return output;
185 }

255 if (HAL_GetTick() - last_pd_calc_time > 50)
256 {
257     last_pd_calc_time = HAL_GetTick();
258
259     // 1. Oblicz nowe PWM na podstawie zmiennej z czujnika
260     float calculated_pwm = Calculate_PD(&robotPD, &distance_sensor.distance_cm);
261
262     // 2. Przekaż do silników
263     current_pwm_output = (int32_t)calculated_pwm;
264     SetRobotSpeed(current_pwm_output);
265 }
266
```

1.3 TEST RESULTS

Testing Procedure:

- Robot positioned at different starting distances.
- Reference value modified remotely.
- Distance evolution observed on LCD and monitoring interface.

Measured parameters:



- steady-state error,
- oscillation behavior,
- response to disturbances.

1.4 CONCLUSION

The implemented system successfully demonstrates closed-loop distance regulation using an STM32-based embedded system. The ultrasonic sensor provided reliable measurements, while PWM motor control enabled smooth movement and accurate positioning.

Advantages observed:

- stable regulation,
- intuitive wireless control,
- clear real-time visualization.

Limitations:

- sensor noise at very small distances,
- mechanical vibrations affecting measurements.

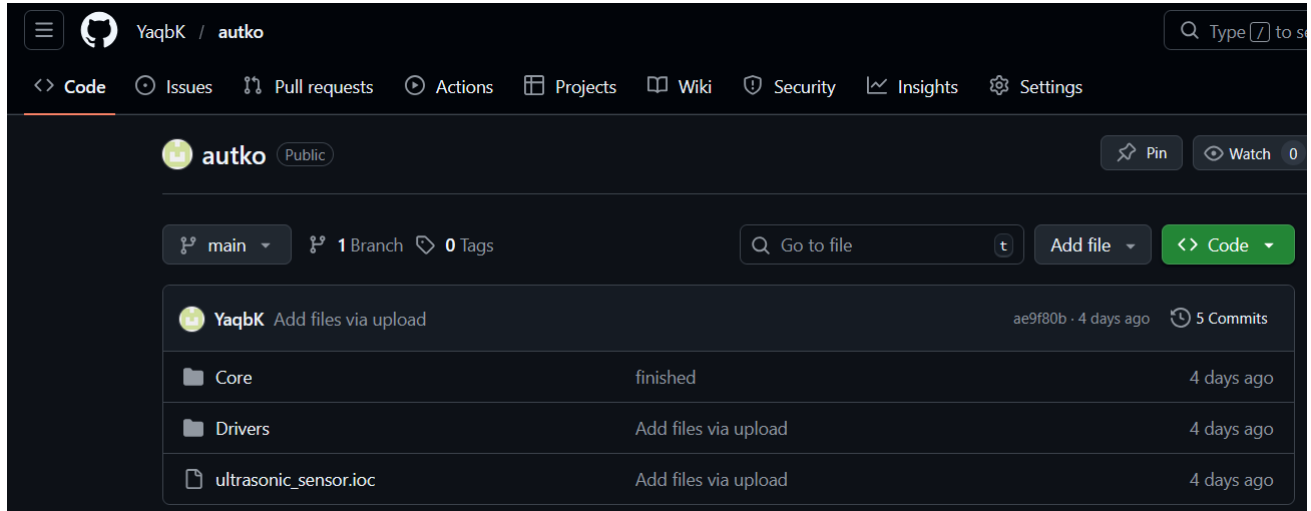
Possible improvements:

- PID parameter tuning,
- signal filtering,
- improved chassis alignment.



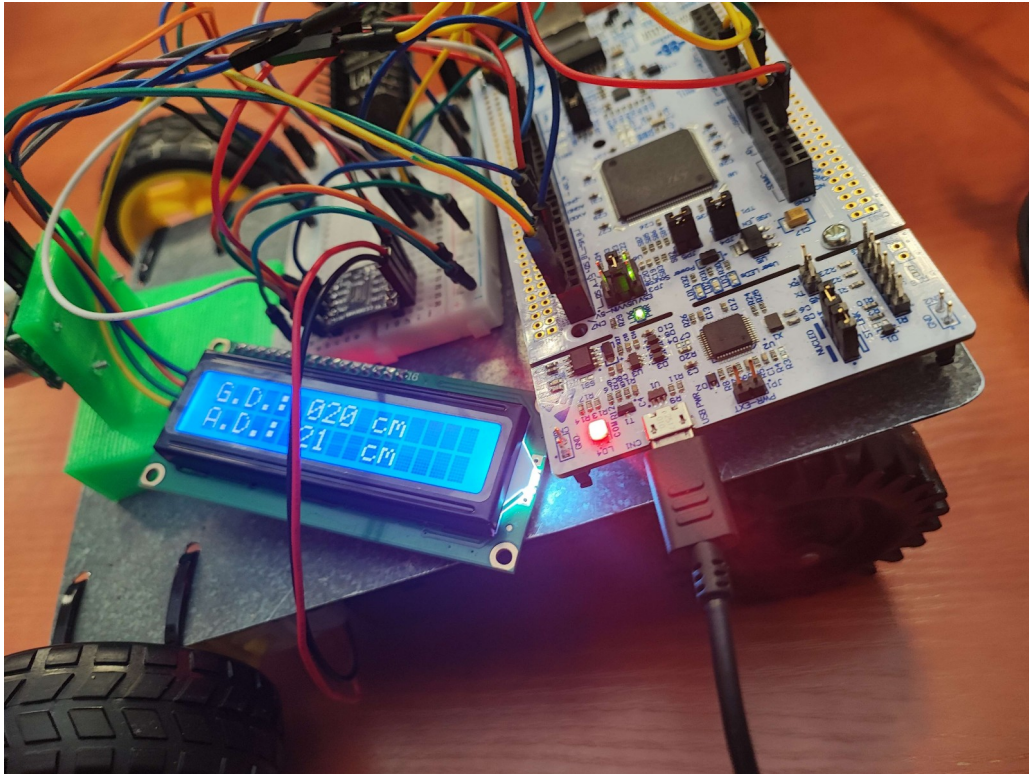
2 ADDITIONAL REQUIREMENTS MET:

2.1 USE OF A VERSION-CONTROL SYSTEM



<https://github.com/YaqbK/autko>

2.2 SYSTEM ENSURES STEADY-STATE ERROR AT 1% OF CONTROL RANGE OR LESS.

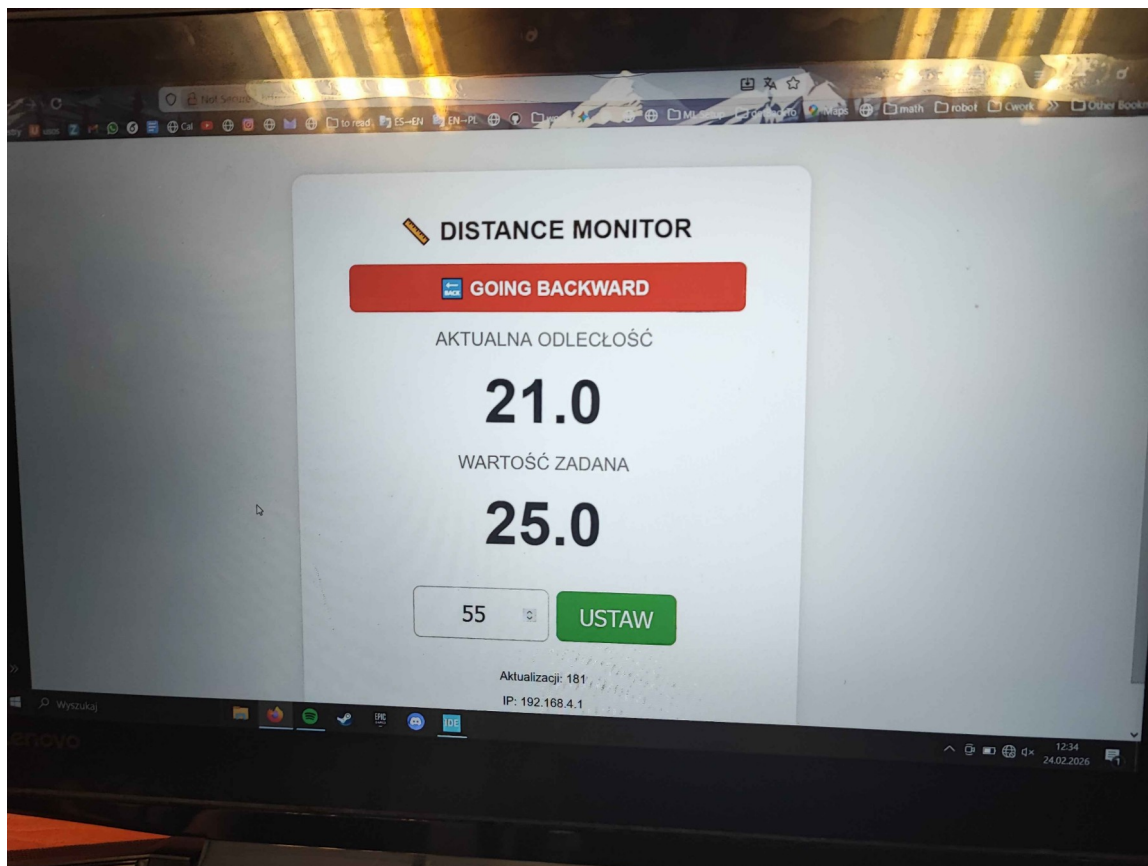


RCW-0001 - ultrasonic distance sensor 4.5 m

Ultrasonic distance sensor RCW-0001 with a measuring range from 1 to 450 cm and a resolution of 1 mm. It is compatible with the popular HC-SR04 sensor. RCW-0001

In the picture it is clearly visible that the given distance was 20 cm, while the achieved distance is 21 cm. This grants a 1cm error with the range of 450cm hence the error itself is equal to $\frac{1}{450} * 100\% = 0.22\%$ which is less than 1%

2.3 DEDICATED DESKTOP APPLICATION AS A GRAPHICAL USER INTERFACE.



A graphical user interface to show the distance monitor as well as allow the user to set the value of the distance set out to be achieved. It was written using arduino code and HTML as snippets above indicate.

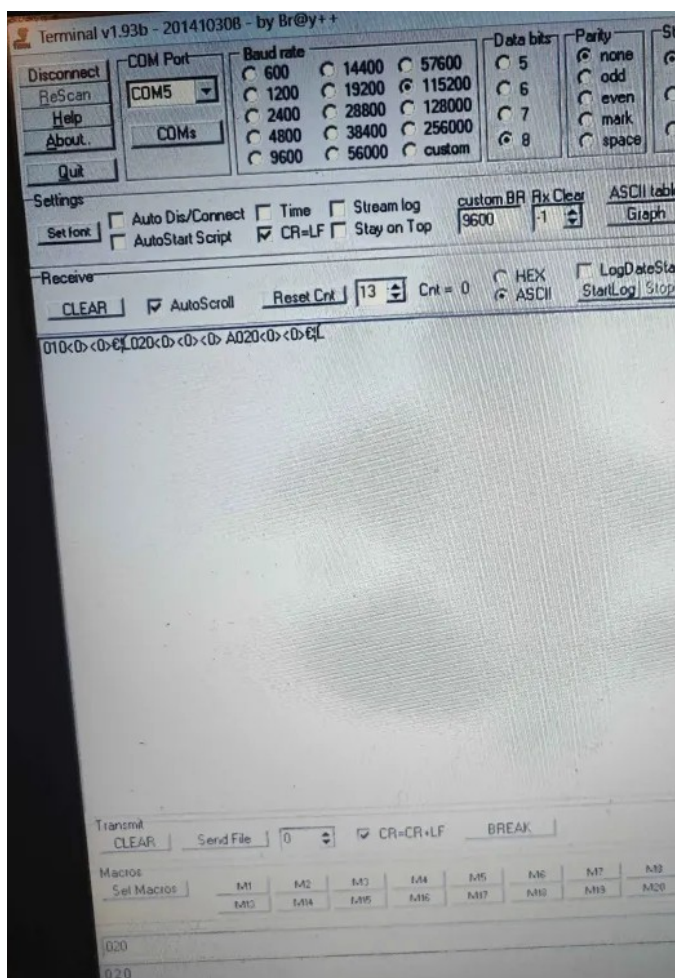
2.4 ADDITIONAL USER OUTPUT DEVICE - LCD DISPLAY.



Clearly visible in the picture LCD display - or an additional user output device.

2.5 ADDITIONAL USER INPUT DEVICE AND USE OF NETWORK COMMUNICATION

Two input sources:



The two sources that allow for inputs to be taken by our project are the terminal inside of the microprocessor board, and the separate wifi module as presented in point no. 3. As mentioned previously we make use of the separate wifi module to communicate with our project. The device creates a separate network that we connect to and allows for remote control of the device.

2.6 NETWORK COMMUNICATION

We used the following separate wifi module, which is ESP8266 NodeMCU v3, that creates its own separate network. After connecting with it we can just open the desired web page and see the output of our program, which is the actual, live distance from the obstacle that is sent from the sensor. But on this page we can also set the new desired distance that we want the vehicle to reach from the obstacle, and it will be sent through this network and module to the vehicle's microcontroller.



ESP8266 NodeMCU v3- wifi module used in the project

Below there is the whole code that was written on the designated app for the wifi module, to create the web page and to execute the distance communication between the microcontroller and the web page for the output visualisation.

```
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3
4  ESP8266WebServer server(80);
5
6  float current_distance = 0;
7  float target_distance = 25.0;
8  uint32_t update_count = 0;
9
10 bool target_changed = false;
11
12 void setup() {
13   Serial.begin(115200);
14   delay(1000);
15
16   WiFi.mode(WIFI_AP);
17   WiFi.softAP("DistanceMonitor", "12345678");
18
19   server.on("/", []() {
20     String html = "<!DOCTYPE html><html><head>";
21     html += "<meta charset='UTF-8'>";
22     html += "<title>Distance Monitor</title>";
23     html += "<style>";
24     html += "body { font-family: Arial; text-align: center; margin-top: 50px; background: #f0f0f0; }";
25     html += ".container { background: white; border-radius: 20px; padding: 30px; margin: 20px auto; max-width: 600px; box-shadow: 0 0 20px rgba(0,0,0,0.1); }";
26     html += ".value { font-size: 80px; font-weight: bold; margin: 20px 0; color: #333; }";
27     html += ".label { font-size: 24px; color: #666; }";
28     html += "input { font-size: 30px; padding: 15px; width: 150px; text-align: center; border: 2px solid #ddd; border-radius: 10px; }";
29     html += "button { font-size: 30px; padding: 15px 30px; background: #4CAF50; color: white; border: none; border-radius: 10px; cursor: pointer; margin-left: 10px; }";
30     html += "button:hover { background: #45a049; }";
```

```
32 html += ".status-box { margin: 20px auto; padding: 15px; border-radius: 10px; font-size: 24px; font-weight: bold; width: 80%; }";
33 html += ".forward { background: #4CAF50; color: white; }";
34 html += ".backward { background: #f44336; color: white; }";
35 html += ".idle { background: #ff9800; color: white; }";
36 html += "</style>";
37
38 html += "<script>";
39 html += "function loadData() {";
40 html += "    fetch('/data')";
41 html += "        .then(response => response.json());";
42 html += "        .then(data => {";
43 html += "            document.getElementById('current').innerHTML = data.distance.toFixed(1);";
44 html += "            document.getElementById('target_display').innerHTML = data.target.toFixed(1);";
45 html += "            document.getElementById('count').innerHTML = data.count;";
46
47 html += "            var statusDiv = document.getElementById('status');";
48 html += "            if (data.distance > data.target + 1) {";
49 html += "                statusDiv.className = 'status-box forward';";
50 html += "                statusDiv.innerHTML = '🚀 GOING FORWARD';";
51 html += "            } else if (data.distance < data.target - 1) {";
52 html += "                statusDiv.className = 'status-box backward';";
53 html += "                statusDiv.innerHTML = '🚗 GOING BACKWARD';";
54 html += "            } else {";
55 html += "                statusDiv.className = 'status-box idle';";
56 html += "                statusDiv.innerHTML = '🎯 TARGET REACHED';";
57 html += "            }";
58 html += "        });";
59 html += "    }";
60 html += "setInterval(loadData, 200);";
61
62 html += "function sendTarget() {";
63 html += "    var val = document.getElementById('target_input').value;";
64 html += "    fetch('/set?val=' + val)";
65 html += "        .then(() => {";
66 html += "            document.getElementById('target_display').innerHTML = parseFloat(val).toFixed(1);";
67 html += "        });";
68 html += "    }";
69 html += "</script>";
70
71 html += "</head><body>";
72 html += "<div class='container'>";
73 html += "<h1>📏 DISTANCE MONITOR</h1>";
74
75 html += "<div id='status' class='status-box idle'>🎯 TARGET REACHED</div>";
76
77 html += "<div class='label'>AKTUALNA ODLEĆAŁOŚĆ</div>";
78 html += "<div class='value' id='current'>" + String(current_distance) + "</div>";
79
80 html += "<div class='label'>WARTOŚĆ ZADANA</div>";
81 html += "<div class='value' id='target_display'>" + String(target_distance) + "</div>";
82
83 html += "<div style='margin: 40px 0;'>";
84 html += "<input type='number' id='target_input' step='0.1' value='" + String(target_distance) + "'>";
85 html += "<button onclick='sendTarget()'>USTAW</button>";
86 html += "</div>";
87
88 html += "<p>Aktualizacji: <span id='count'>" + String(update_count) + "</span></p>";
89 html += "<p>IP: " + WiFi.softAPIP().toString() + "</p>";
90 html += "</div>";
91 html += "</body></html>";
92
93 server.send(200, "text/html", html);
```

```
96  server.on("/set", []() {
97      if (server.hasArg("val")) {
98          target_distance = server.arg("val").toFloat();
99          target_changed = true;
100      }
101      server.send(200, "text/plain", "OK");
102  });
103
104  server.on("/data", []() {
105      String json = "{";
106      json += "\"distance\":" + String(current_distance) + ",";
107      json += "\"target\":" + String(target_distance) + ",";
108      json += "\"count\":" + String(update_count);
109      json += "}";
110      server.send(200, "application/json", json);
111  });
112
113  server.begin();
114 }
```

Loop that is responsible to control the whole idea of the project, which is the distance communication. It is sending the chosen distance on the web page, through the network and wifi module to the microcontroller and at the same time sending the actual, live distance read from the sensor through the microcontroller to wifi module and to the web page for the users to see the output:.

```
116 void loop() {
117     server.handleClient();
118
119     // Receiving thr distance from STM32
120     if (Serial.available()) {
121         String data = Serial.readStringUntil('\n');
122         data.trim();
123
124         if (data.length() > 0) {
125             float val = data.toFloat();
126             if (val > 0 || data == "0.0") {
127                 current_distance = val;
128                 update_count++;
129             }
130         }
131     }
132
133     // Sending the target distance to STM32
134     if (target_changed) {
135         char buf[3];
136         sprintf(buf, "%f", target_distance);
137         Serial.write(buf);
138
139         target_changed = false;
140     }
141
142     delay(10);
143 }
```

```
111 float uint8_to_float_ascii(uint8_t bytes[3]) {
112     // Assuming bytes contain ASCII digits: '3', '0', '0'
113     if(bytes[0] != '.' && bytes[1] != '.' && bytes[2] != '.'){
114         int value = (bytes[0] - '0') * 100 +
115                     (bytes[1] - '0') * 10 +
116                     (bytes[2] - '0');
117         return (float)value;
118     }
119     else if(bytes[2] == '.'){
120         int value = (bytes[0] - '0') * 10 + (bytes[1] - '0') ;
121         return (float)value;
122     }
123     else if(bytes[1] == '.'){
124         int value = (bytes[0] - '0');
125         return (float)value;
126     }
}
```

```
334 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
335 {
336     if(TIM1 == htim->Instance)
337     {
338         uint32_t echo_us;
339
340         echo_us = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);
341         distance_sensor.distance_cm = hc_sr04_convert_us_to_cm(echo_us);
342     }
343 }
344
```

```
345 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
346 {
347     /* Prevent unused argument(s) compilation warning */
348     UNUSED(huart);
349
350     /* NOTE : This function should not be modified, when the callback is needed,
351              the HAL_UART_RxCpltCallback can be implemented in the user file.
352     */
353     HAL_UART_Transmit(&huart3, rx_data, 8, 10);
354 }
355
```

Part of while loop responsible for wifi connection:



```
268
269     distances = (float)distance_sensor.distance_cm;
270
271     char wifi_buffer[50];
272     sprintf(wifi_buffer, "%.1f\n", distances);
273     HAL_UART_Transmit(&huart2, (uint8_t*)wifi_buffer, strlen(wifi_buffer), 1000);
274
275
```

3. REFERENCES

1. STM32 HAL and CMSIS documentation.
2. DRV8833 Motor Driver Datasheet.
3. RCWL0001 Ultrasonic Sensor Documentation.
4. Course materials – Microprocessor Systems Laboratory.